판다스 치트시트 — 빅데이터분석기사 실기 대비

예시 표 — df: 별빛 분식 메뉴판(메뉴·분류·가격·칼로리·매운맛·판매량) · o: 주문 기록(날짜·시각·메뉴·수량·결제) · raw: 정리 안 된 메뉴판. ★ 꼭 외우기 · ⚠ 함정

판다스 실습실 · © 2026 티쳐무
코드는 pandas 2.2 로 모두 실행해 확인함

0. 시작 · 파일

```
import pandas as pd
import numpy as np
```

★ 첫 두 줄. 이후 pd. · np. 으로 부른다

```
df = pd.read_csv('menu.csv')
```

CSV 읽기 — 시험에서는 문제가 준 경로 그대로

```
pd.read_csv('menu.csv', index_col=0)
```

첫 열을 이름표로 (Unnamed: 0 막기)

```
pd.read_csv('orders.csv', parse_dates=['날짜'])
```

읽으면서 날짜 자료형으로

```
df.to_csv('result.csv', index=False)
```

★ 저장 — 작업형2 제출은 index=False

```
pd.DataFrame({'메뉴': ['김밥', '라면'], '가격': [3500, 4000]})
```

표 만들기 {열이름: [값들]}

```
pd.Series([3500, 4000], index=['김밥', '라면'])
```

시리즈 만들기 (이름표 붙이기)

1. 첫 확인 (EDA)

```
df.head()  ·  df.tail(3)
```

앞 5줄 · 뒤 3줄

```
df.shape
```

★ (행, 열) — 괄호 없음! shape[0] 이 행 수

```
df.info()
```

줄 수 · 열마다 non-null 개수 · dtype

```
df.describe()
```

숫자 열 요약
(count·mean·std·min·25%·50%·75%·max)

```
df.describe(include='object')
```

글자 열 요약 (unique·top·freq). 'O' 도 됨

```
df.dtypes  ·  df.columns
```

열마다 자료형 · 열 이름 목록

```
df['분류'].value_counts()
```

★ 값마다 개수 — counts 의 s!

```
df['분류'].value_counts(normalize=True)
```

비율로

```
df['분류'].unique()  ·  df['분류'].nunique()
```

종류 목록 · 종류 수

```
df.corr(numeric_only=True)
```

⚠ 상관계수 — 글자 열이 있으면 numeric_only=True

2. 고르기

```
df['가격']  ·  df[['메뉴', '가격']]
```

★ 한 겹 = 시리즈 · 두 겹 = 표

```
df.loc[1:3, '메뉴':'가격']
```

★ loc = 이름표·열 이름, 끝 포함 (1·2·3)

```
df.iloc[1:3, 0:2]
```

★ iloc = 위치 번호, 끝 미포함 (1·2)

```
df.loc[df['가격'] > 5000, '메뉴']
```

조건에 맞는 줄의 열

```
cond = (df['가격'] >= 4000) & (df['매운맛'] > 0)
df[cond]
```

★ & 그리고 · | 또는 · ~ 아님 — 조건마다 괄호

```
df[df['분류'].isin(['음료', '디저트'])]
```

목록 안에 있나 — 리스트로

```
df[df['가격'].between(3000, 5000)]
```

범위 (양 끝 포함)

```
df[df['메뉴'].str.contains('김밥', na=False)]
```

글자가 들어 있나 (빈칸은 False 로)

```
i = df['판매량'].idxmax()
df.loc[i, '메뉴']
```

⚠ 최댓값 줄의 다른 열 — iloc 에 열 이름 X

3. 통계 · 개수

```
df['가격'].mean()  # median std var sum min max count
```

std·var 는 n−1 로 나눈다 (ddof=0 이면 n)

```
df['가격'].quantile(0.25)
```

1사분위수 · 0.75 는 3사분위수

```
df['분류'].mode()[0]
```

★ 최빈값 하나 — mode() 는 시리즈라 [0]

```
(df['가격'] > 5000).sum()
```

★ 조건에 맞는 개수 (True=1). len(df[cond]) 도 같음

```
(df['가격'] > 5000).mean()
```

조건에 맞는 비율

```
df.nlargest(3, '판매량')
```

가장 큰 3줄 (작은 것은 nsmallest)

```
df.sum(numeric_only=True)
```

axis=0(기본) 열마다 · axis=1 줄마다

```
df['가격'].cumsum()
```

누적 합

4. 정렬

```
df.sort_values('판매량', ascending=False)
```

★ 값 정렬 — values 의 s! 큰 것부터는 False

```
df.sort_values(['분류', '가격'], ascending=[True, False])
```

여러 열은 리스트로 짝지어

```
df = df.sort_values('칼로리').reset_index(drop=True)
```

⚠ 대입해야 남는다 · drop=True 로 옛 이름표 버림

```
df.sort_index()
```

이름표 차례로

5. 바꾸기 · 만들기

```
df['매출'] = df['가격'] * df['판매량']
```

새 열 = 계산

```
df.drop(columns=['칼로리'])  ·  df.drop('칼로리', axis=1)
```

열 지우기 (행은 drop(이름표))

```
df.rename(columns={'가격': '판매가'})
```

열 이름 바꾸기

```
df['가격'].astype(float)
```

자료형 바꾸기 (int · float · str)

```
df['분류'].replace('식사', '밥')
```

칸 전체가 같을 때 바꾸기 (일부는 str.replace)

```
df['분류'].map({'분식': 1, '식사': 2, '음료': 3, '디저트': 4})
```

짝 사전으로 — 사전에 없으면 NaN

```
df['칼로리'].apply(lambda x: '높음' if x >= 500 else '낮음')
```

칸마다 함수 (def 함수도 됨)

```
np.where(df['칼로리'] >= 500, '높음', '낮음')
```

조건 참·거짓으로 두 값 중 하나

```
df.loc[df['분류'] == '음료', '가격'] = 1800
```

조건에 맞는 칸만 값 바꾸기

```
pd.cut(df['가격'], bins=[0, 3000, 6000, 9000], labels=['싸다', '보통', '비싸다'])
```

구간 나누기 (오른쪽 끝 포함)

6. 결측치 · 중복

```
raw.isnull().sum()
```

★ 열마다 빈칸 수 (isna 와 같다). .sum().sum() 은 전체

```
raw.dropna(subset=['가격'])
```

빈칸 있는 줄 지우기 (subset 없으면 아무 열이나)

```
raw['칼로리'].fillna(raw['칼로리'].median())
```

중앙값으로 채우기 (평균은 mean)

```
raw['원산지'].fillna(raw['원산지'].mode()[0])
```

글자 열은 최빈값으로

```
raw['칼로리'].fillna(raw.groupby('분류')['칼로리'].transform('mean'))
```

★ 무리마다 평균으로 채우기

```
raw['칼로리'].ffill()
```

앞 값으로 (뒤 값은 bfill)

```
raw.duplicated().sum()  ·  raw.drop_duplicates()
```

중복 줄 수 · 지우기

```
pd.to_numeric(raw['맵기'], errors='coerce')
```

숫자로 — 못 바꾸는 값은 NaN

7. 문자열 (.str)

```
df['메뉴'].str.replace('떡볶이', '떡')
```
★ 칸 안의 일부 글자 바꾸기

```
df['메뉴'].str.contains('김밥')
```
들어 있나 (여러 개는 '김밥|라면')

```
o['시각'].str.split(':').str[0]
```
나눠서 칸마다 첫 조각 (마지막은 str[-1])

```
df['메뉴'].str[0:2]
```
⚠ 칸마다 글자 자르기 — str 빼면 줄 고르기!

```
df['메뉴'].str.len()
```
글자 수

```
df['메뉴'].str.strip().str.lower()
```
앞뒤 빈칸 지우기 · 소문자 (upper 대문자)

```
df['메뉴'].str.startswith('치즈')
```
~로 시작하나 (endswith)

8. 날짜 · 시계열 (.dt)

```
o['날짜'] = pd.to_datetime(o['날짜'])
```
★ 글자 → 날짜 자료형

```
pd.to_datetime(o['날짜']).dt.month
```
dt.year · month · day · hour · quarter

```
pd.to_datetime(o['날짜']).dt.dayofweek
```
⚠ 요일 번호 — 월 0 ~ 일 6 (주말: >= 5)

```
pd.to_datetime(o['날짜']).dt.day_name()
```
요일 이름 (Monday ...)

```
o[pd.to_datetime(o['날짜']) >= '2025-03-07']
```
날짜는 글자와 바로 비교

```
pd.to_datetime('2025년03월07일', format='%Y년%m월%d일')
```
★ 낯선 모양은 format — %Y 4자리·%y 2자리 ·%m 월·%d 일·%H 시·%M 분·%S 초·%b Mar·%p AM/PM

```
pd.to_datetime('2025-03-07') + pd.Timedelta(days=99)
```
⚠ 100일째 = +99일 · 이름은 weeks·days·hours·minutes·seconds (s!)

```
t = pd.to_datetime(o['날짜'] + ' ' + o['시각'])
(t - t.min()).dt.days
```
날짜 − 날짜 = 기간(timedelta) · 일 수는 dt.days

```
t = pd.to_datetime(o['날짜'] + ' ' + o['시각'])
(t - t.min()).dt.total_seconds() / 60
```
⚠ 기간 → 분은 total_seconds()/60 — dt.seconds 는 하루 뺀 나머지 초

```
pd.to_datetime(o['날짜']).dt.to_period('M')
```
월·분기로 묶기 'Y'·'Q'·'M'·'D'·'h' (옛 'A'·'H' 는 경고)

```
print(round(2.5), round(3.5))
```
⚠ 파이썬 round 는 딱 .5 이면 짝수 쪽 → 2 4

9. 묶기 · 피벗

```
df.groupby('분류')['가격'].mean()
```
★ groupby('무리')['열'].계산() — 함수 ( ) · 열 [ ]

```
df.groupby('분류').size()
```
무리마다 줄 수

```
df.groupby('분류')['가격'].agg(['mean', 'max'])
```
여러 계산 한 번에

```
df.groupby('분류').agg({'가격': 'mean', '판매량': 'sum'})
```
열마다 다른 계산

```
df.groupby(['분류', '매운맛'])['판매량'].sum().reset_index()
```
두 열로 묶고 평평한 표로 (as_index=False 도 됨)

```
df.groupby('분류')['가격'].mean().idxmax()
```
평균이 가장 큰 무리의 이름

```
o.pivot_table(index='날짜', columns='결제', values='수량', aggfunc='sum', fill_value=0)
```
★ 줄·열·값·계산 — aggfunc 기본은 mean

```
o.groupby(['날짜', '결제'])['수량'].sum().unstack()
```
안쪽 단계를 열로 (level=0 이면 바깥)

```
pd.melt(df, id_vars=['메뉴'], value_vars=['가격', '칼로리'], var_name='항목', value_name='값')
```
넓은 표 → 긴 표

```
pd.crosstab(o['날짜'], o['결제'])
```
두 열의 빈도표

10. 합치기

```
pd.concat([df.head(2), df.tail(2)], ignore_index=True)
```
★ 위아래로 잇기 · 이름표 새로

```
pd.concat([df[['메뉴']], df[['가격']]], axis=1)
```
좌우로 (이름표끼리 짝)

```
pd.merge(o, df[['메뉴', '가격']], on='메뉴', how='left')
```
★ 열쇠로 짝짓기 — how='inner'(기본)·left·right·outer

11. 작업형1 단골 꼴

```
s = df['판매량'].sort_values(ascending=False)
df.loc[s.index[:3], '판매량'] = s.iloc[2]
print(df['판매량'].mean())
```
상위 n개를 n번째 값으로 바꾸기

```
q1, q3 = df['가격'].quantile([0.25, 0.75])
iqr = q3 - q1
print(((df['가격'] < q1 - 1.5 * iqr) | (df['가격'] > q3 + 1.5 * iqr)).sum())
```
IQR 이상치 개수

```
x = df['가격']
print(((x - x.min()) / (x.max() - x.min()) > 0.5).sum())
```
최소-최대 정규화 (0~1)

```
x = df['가격']
print(((x - x.mean()) / x.std()).max())
```
표준화 (z 점수)

```
b = raw['칼로리'].std()
a = raw['칼로리'].fillna(raw['칼로리'].median()).std()
print(round(b - a, 2))
```
채우기 전후 표준편차 차이

```
print(int(df['가격'].mean()))  # 버림
print(round(df['가격'].mean(), 2))  # 반올림
```
⚠ 정수 부분만 · 소수 둘째 자리 반올림

12. 시험장에서

```
print(답)
```
★ 작업형1 은 출력한 값으로 채점 — 마지막 줄만 쓰면 안 나온다

```
help(pd.DataFrame.fillna)  ·  dir(pd)
```
인자 이름이 생각 안 나면 도움말·목록 보기

```
df = df.dropna()
```
⚠ dropna·fillna·sort_values·replace 는 대입해야 남는다

```
and / or → & / |  ·  shape() → shape  ·  value_count → value_counts
```
⚠ 자주 틀리는 철자·문법

같은 값끼리의 정렬 순서

⚠ 정렬·value_counts 의 동점 순서는 컴퓨터마다 다를 수 있다 — 동점이 있는지 확인

시험 형식 · 배점

회차마다 바뀔 수 있다 — 한국데이터산업진흥원 데이터자격검정(dataq.or.kr) 공고 확인

© 2026 티쳐무 · 모든 권리 보유. 학교 수업과 개인 공부 목적으로 자유롭게 인쇄해 쓰세요. 재배포·수정·상업적 이용은 먼저 문의해 주세요. 시험장에는 가지고 들어갈 수 없으니 외워 두세요.